Architecture Decisions

AI Literacy for the Modern Workforce

This architecture decision record lists every technical decision behind the custom platform. The decisions were recorded before implementation began, so the AI development partner had a single reference to build against. Expand any card for the decision and its rationale.

The decisions share two aims, simplicity and openness, and together they keep the whole platform a single, self-contained site anyone can open, read, and check.

9 decisions

FRAMEWORK

Vite + React + TypeScript (strict mode)

ADOPTED

The platform is built with three common web tools. React is a widely used framework for building interactive interfaces, and it works well with the charting and interface libraries the dashboards and sandbox rely on. Vite is the build tool that packages everything the browser downloads, and it keeps that package small and quick to load.

TypeScript is a version of JavaScript that checks the code for mistakes before the site ever ships, rather than letting them surface for a learner. Its strict setting applies those checks more rigorously, which is especially useful where the research data files connect to the charts that display them.

ROUTING

React Router v6 with HashRouter

ADOPTED

Moving between the four modules and their sections runs on React Router, a common navigation tool for a site like this. It is set to "hash" mode, which is why every address carries a # (for example, /#/module/1/section/2). That # keeps the part after it on the visitor's device instead of sending it to a server, so the site runs on simple, free hosting with no server setup.

STYLING

Tailwind CSS with design tokens in the config

ADOPTED

The visual styling uses Tailwind, a system that builds the look from small, reusable pieces. It keeps the design consistent across the site and strips out any styling the site does not actually use, so the styling file the browser downloads stays tiny.

Every choice from the earlier design system (the fonts, the four competency colors and their variants, the spacing, the corner rounding, the screen-size breakpoints) is defined once in a central settings file, so the whole platform draws from one source and stays consistent.

CHARTING

Recharts

● ADOPTED

Module 2's dashboard shows several interactive charts: the split between AI augmenting and automating tasks, adoption by country, productivity gains, and projected skill demand. Recharts is a charting library made for React, and it draws all of these with very little custom code. An alternative, D3, offers more control but would have added a lot of extra work the charts here do not need.

STATE MANAGEMENT

React Context + useReducer, persisted to localStorage

● ADOPTED

The platform only needs to remember a small, well-defined amount of information, so it uses React's built-in tools for this rather than a separate add-on library. Two pieces cover everything.

The first remembers each learner's progress: where they are, which sections they have finished, their knowledge-check answers, and their written reflections, all saved in the browser so nothing is lost on a refresh or an interruption. The second records activity for the analytics dashboard and can export it as a file the organization can take with it.

DATA APPROACH

Static JSON in /src/data, no backend for V1

● ADOPTED

All the data behind the charts comes from published research with fixed numbers, so it never changes while the site is running and needs no server to process it. It is stored in plain data files that load straight into the page, which is fast and keeps hosting simple.

Those files are also open to inspection: anyone can open one and check that a figure traces back to its source study, such as Handa et al. (2025). Learner activity is saved in the browser and can be exported as a file, so none of this requires a server.

DEPLOYMENT

Cloudflare Pages (primary), GitHub Pages (fallback)

● ADOPTED

The site is hosted on Cloudflare Pages. The domain was already managed by Cloudflare, so this setup handles the web address automatically, and its network delivers pages quickly across Southeast Asia at no cost. Each time an update is saved, the site rebuilds and publishes itself, and draft versions get their own preview links.

GitHub Pages is a free backup host that needs no special configuration (the hash-based addresses above are what make that possible). Its network is a little slower than Cloudflare's, a fair tradeoff for a low-traffic portfolio site.

EVALUATED AND DECLINED

NETLIFY

Evaluated and declined

DECLINED

Netlify would have done the same job as Cloudflare Pages: free hosting, automatic publishing on save, and custom-domain support. It was declined because it would mean adding another company's account with no real advantage over Cloudflare, which already manages the domain.

VERCEL

Evaluated and declined

DECLINED

Declined on security grounds. In April 2026, Vercel disclosed a serious breach: an attacker got into a third-party AI tool used by a Vercel employee, took over that employee's work account, and reached internal systems, including customer configuration data the attacker was able to decrypt. The stolen data was put up for sale online.

This site stores no sensitive data of its own, so it was not directly at risk, but the breach pointed to a broader security concern. Vercel was also unnecessary here, since Cloudflare already manages the domain.

The standards behind them

Every decision here was held to the same fixed standards during the build: the code is checked thoroughly for mistakes and held to a clean browser console; the platform is built to the WCAG 2.1 AA accessibility standard as a baseline; every page is structured to support screen readers and other assistive tools; and every file opens with a short comment explaining what it does, with data files citing the research they draw from. This page and the seven others in this series were built to those standards.

© Michael Ritchot. Code licensed under the MIT License. Instructional content licensed under CC BY-NC-SA 4.0.